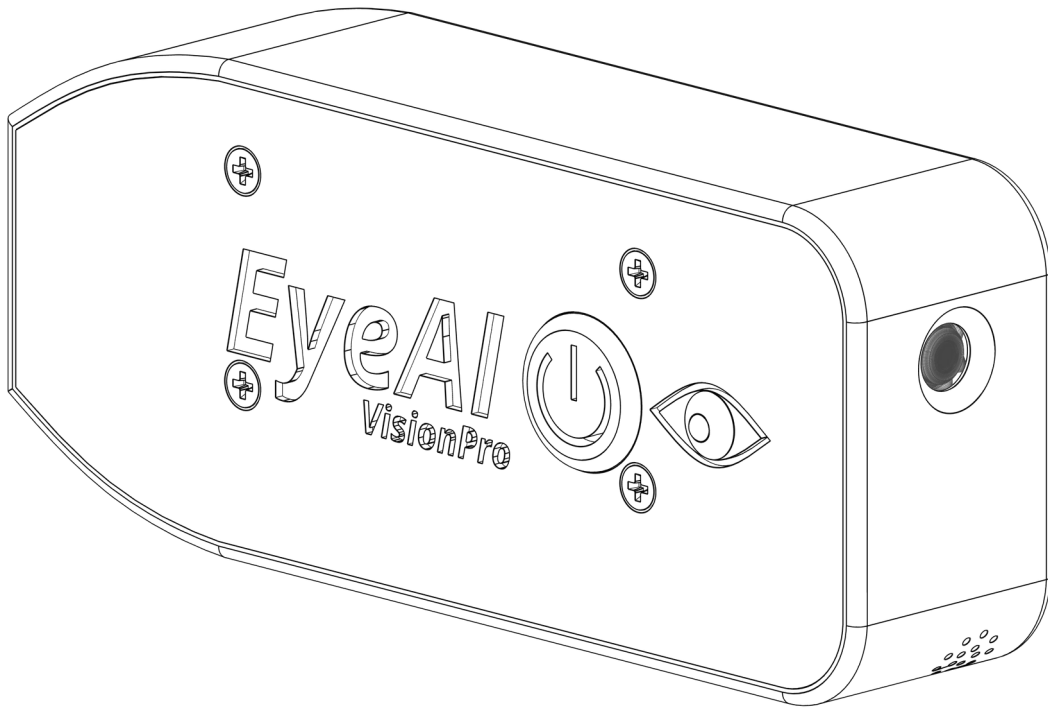


Projektdokumentation

EyeAIVisionPro



Bearbeiter: Thomas Fritzler

Ostfildern, der 18.09.2026

Inhaltsverzeichnis

Motivation.....	3
Anforderungen.....	3
Konzept.....	4
Schaltungsdesign.....	5
Gehäusedesign.....	7
Software.....	8
Praktische Verwendung der EyeAIVisionPro.....	10
Bekannte Probleme.....	10
Technische Daten.....	10
Fazit.....	11

Motivation

Das Ziel des Projekts EyeAI ist es, für blinde Personen eine Sehhilfe zu schaffen, welche mittels Computer-Vision-Algorithmen die Umgebung erfasst und diese per Audioausgabe beschreibt. Damit diese Datenaufnahme sowohl aus Perspektive des Nutzers als auch ohne das händische Bedienen eines Mobiltelefons erfolgen kann, haben wir uns dazu entschieden, zusätzlich zu unserer Android-App eine Hardware zu entwickeln: Die EyeAIVision. Diese ist ein Gerät im Formfaktor eines Brillenadapters, welcher sich möglichst einfach am Brillenbügel montieren lässt. Es fungiert als eine ESP32-gestützte Kamera, welche Bilddaten mit einem OV2640-Kameramodul aufnimmt und mittels MJPEG-Stream an das Mobiltelefon des Nutzers übermittelt. Dort werden die aufgenommenen Bilder in Echtzeit verarbeitet.

Zusätzlich dazu enthält die EyeAIVision einen Touch-Button (welcher die metallische Fassung der Status-LED ist), mit dem sich die in die App eingebaute Sprachsteuerung bedienen lässt. In der Praxis hat sich jedoch herausgestellt, dass das Gerät aufgrund eines schwachen WLAN-Moduls sowie des ineffizienten MJPEG-Streams häufige Verbindungsprobleme hat. Zusätzlich muss EyeAIVision sich als Wi-Fi-Station mit einem Access Point des Mobiltelefons (welcher zusätzlich manuell vom Nutzer erstellt werden muss) verbinden, was sich negativ auf die Nutzerfreundlichkeit auswirkt.

Aus diesen Gründen entschieden wir uns dazu, die nächste Generation von EyeAIVision zu entwickeln: Die EyeAIVisionPro. Diese soll dem Grundprinzip eines Brillenadapters folgen. Zudem sollen die genannten Stabilitätsprobleme gelöst sowie weitere Features hinzugefügt werden.

Anforderungen

Zu den Anforderungen an das Gerät gelten:

- Stabile Bildübertragung mit niedriger Latenz
- Erzeugung eines eigenen WLAN Access Points
- Integrierter Akkumulator mit Lademöglichkeit per USB-C
- Ein-, Ausschalt- und Funktionstaste (anstatt Schiebeschalter)
- Komfortabel zusammenzubauendes Gerät
- Rot/grüne Status-LED
- Eingebauter Kopfhörerausgang
- Eingebautes Mikrofon
- Messbarer Batteriestatus
- Integrierter Lautsprecher

Konzept

Zur Realisierung einer stabilen Bildübertragung war der wichtigste Schritt die Abkehr vom ESP32-Mikrocontroller. Dieser ist sowohl von der Prozessorleistung als auch im Bezug auf das eingebaute WLAN-Modul weitaus zu schwach für unsere Anforderungen. Stattdessen verwenden wir den Raspberry Pi Zero W als Computer für das Gerät. Der Einplatinencomputer besitzt einen BCM 2835 SoC (Single-Core) mit 512 MB Arbeitsspeicher sowie On-Board Wireless LAN und Bluetooth. Zudem unterstützt er hardwarebasiertes H.264-Videoencoding und eine CSI-Schnittstelle. Diese unterstützt von Werk aus das Raspberry Pi Camera Module 3, das einen leistungsstärkeren IMX708 12-Megapixel-Sensor von Sony enthält.

Zur Realisierung der Stromversorgung, der Taste, eines einfachen Batteriestandsmessers und der Status-LED haben wir uns für ein eigens entwickeltes HAT (Hardware Attached on Top) entschieden. Der Vorteil von diesem ist vor allem die Flexibilität mit anderen Komponenten der EyeAIVisionPro. So soll das HAT mit wenig Aufwand auf die 40-Pol-Buchsenleiste des Raspberrys gesteckt werden können, während weitere Komponenten mittels Steckverbindung verbunden werden können. Dies erleichtert die Wartung des Geräts im Vergleich zum Vorgänger erheblich.

Ein zusätzliches Feature der EyeAIVisionPro ist der eingebaute Kopfhöreranschluss. Somit soll die Nutzung von separaten (Bluetooth-)Kopfhörern nicht mehr notwendig sein. Es können simple Kopfhörer mit 3,5-mm-Klinkenanschluss verwendet werden, um die Tonausgaben der EyeAIApp hören zu können. Da der Raspberry Pi Zero W keine integrierte Soundkarte besitzt, muss die Tonausgabe über ein externes I2S-Codec-Board erfolgen. Wir haben uns für das WM8960 Audio Codec Breakout von SparkFun entschieden, da dieses neben einem Kopfhörerausgang unter anderem auch einen Mikrofoneingang und Lautsprecherausgänge besitzt.

Für die Stromversorgung wird weiterhin ein Lithium-Ionen-Akku mit 3,7 V Nennspannung und einer Kapazität von 1500 mAh verwendet. Im Vergleich zum Vorgänger muss dieser jedoch außerhalb des Hauptgehäuses liegen, da dieses nicht groß genug für ihn wäre.

Zusammengehalten wird die Elektronik von einem 3D-Druck-PLA-Gehäuse, in welchem alle Bauteile entweder festgeschraubt oder festgeklemt werden sollen. An den Außenseiten befinden sich Buchsen für das Aufladen sowie den Audioausgang als auch eine Kameraaussparung, die Fassung für die Status-LED und eine große Kunststofftaste für Funktionen und das Starten des Geräts.

Schaltungsdesign

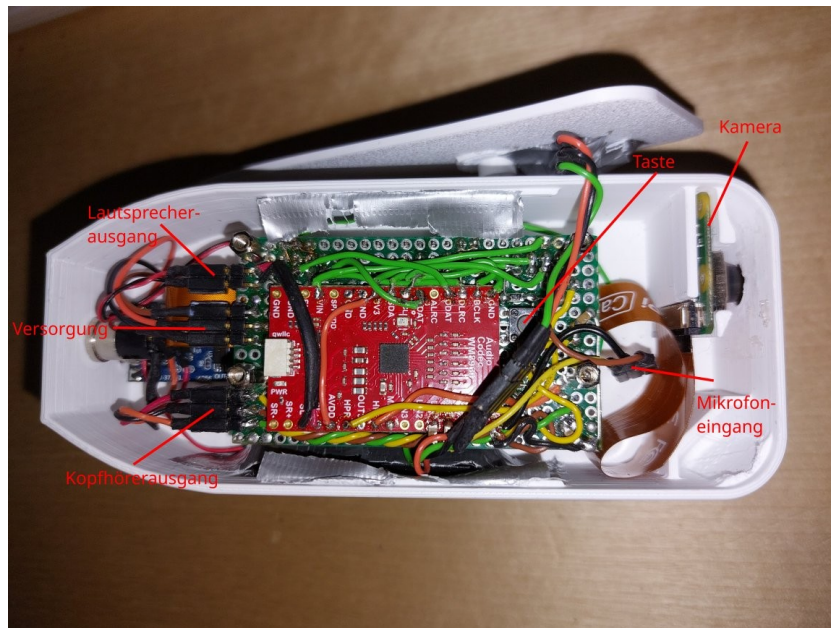


Abbildung 1: Innenansicht auf die Elektronik

Die Schaltung muss als HAT auf den Raspberry Pi steckbar sein, um eine saubere Montage zu ermöglichen. Dies wurde (wie bei allen anderen HAT-Modulen auch) durch das Anbringen einer 40-Pol-Pinleiste umgesetzt. Somit war es uns möglich, jeden GPIO-Pin des Rasperrys mit unserer Schaltung verbinden zu können.

Der wichtigste Aspekt unserer Schaltung ist jedoch die akkubetriebene Stromversorgung. Diese haben wir mit einem TP4056-Lademodul und einem zugehörigen 1500-mAh-Lithium-Ionen-Akku umgesetzt. Das Problem dieses Moduls ist es jedoch, dass ein gleichzeitiges Aufladen und Benutzen des Akkus diesen auf lange Sicht beschädigen können. Aus diesem Grund mussten wir eine Bypass-Schaltung konstruieren. Diese trennt mittels P-MOSFET die Verbindung zwischen Akku und der restlichen Schaltung, sobald die 5 Volt des USB-C-Ladegeräts angelegt werden. In diesem Fall wird die gesamte Schaltung vom Ladegerät versorgt, welches gleichzeitig über das TP4056 den Akkumulator auflädt.

Im Vergleich zum Vorgängermodell besitzt die EyeAIVisionPro anstatt eines Mikrocontrollers einen Raspberry Pi, welcher seitens des Betriebssystems heruntergefahren werden muss, um ein mögliches Beschädigen dieses zu vermeiden. Aus diesem Grund ist es nicht mehr sinnvoll, einen Schiebeschalter zwischen Batterie und Verbraucher zu schalten, der die gesamte Stromversorgung unterbrechen kann. Stattdessen verwenden wir eine On-Off-Button-Schaltung, mit welcher sich die Stromversorgung ein- und ausschalten lässt. Wird dieser gedrückt, so wird eine Verbindung zwischen Batterie und Verbraucher kurzfristig hergestellt. Dieses kurze Zeitfenster reicht aus, um einen auf der Platine integrierten ATtiny85-Mikrocontroller zu starten. Liegt an diesem eine Spannung an, so wird der physische Pin PB1 auf „HIGH“ gesetzt, wodurch T2, ein BC547-PNP-Transistor, durchschaltet. Somit fällt zwischen Gate und Drain des P-MOSFETs T1 eine Spannung

ab, wodurch dieser durchschaltet. Dadurch erhält die ganze Schaltung über T1 Strom, bis T2 nicht mehr durchschaltet. Diese Funktionalität wird durch den ATtiny85 geregelt, welcher als I2C-Slave vom Raspberry Pi gesteuert werden kann.

Da derselbe Button auch als Funktionstaste und Ausschalttaste verwendet werden soll, muss dieser durch den Raspberry Pi auslesbar sein. Deshalb enthält die Schaltung eine Kombination aus zwei Spannungsteilern, an welchen abhängig vom Tasterstatus unterschiedliche Spannungen anliegen. Wird der Taster gedrückt, so fließt ein Strom über R5, R3 und den Taster zu GND. Somit fällt keine Spannung an R6 ab, welcher vom Raspberry Pi als Indikator für den Button-Status verwendet wird. Somit wird eine logische „0“ ausgelesen, welche als „Schalter gedrückt“ interpretiert wird. Bei offenem Schalter fließt der Strom nicht mehr über R3, sondern über R6, wodurch an diesem eine Spannung von etwa 3,3 Volt abfällt, welche als logische „1“, also „Schalter offen“, interpretiert wird.

Damit der Status der EyeAIVision visuell angezeigt werden kann, enthält die Schaltung Lötunkte für eine zweifarbige Status-LED. Um diese direkt nach dem Start auf Rot zu setzen, wird diese ebenfalls vom ATtiny85 gesteuert. Wird Pin PB3 des ATtiny85 auf „HIGH“ gestellt, so schaltet der Transistor T4 nicht mehr durch, wodurch die rote LED erlischt, während die grüne eingeschaltet wird.

Zur Abschätzung der Batterieladung befindet sich ein weiterer Spannungsteiler bestehend aus R12 und R13 auf der Platine. Da der Raspberry Pi keinen integrierten Analog-Digital-Converter besitzt, muss der ATtiny85 diese Messwerte aufnehmen. Ausgelesen werden diese über den I2C-Bus. Auch bei ausgeschaltetem Gerätezustand ist die Batterie über den Spannungsteiler mit GND verbunden. Deshalb mussten wir diese relativ hoch dimensionieren, um hohe Ströme und somit Verlustleistungen zu vermeiden.

Um den Raspberry Pi mit konstanten 5 Volt versorgen zu können, befindet sich auf der Platine ein externer Step-Up-Wandler, welcher die Batteriespannung, welche sich je nach Ladestand zwischen 3,5 V und 4,2 V befindet, auf die nötigen 5 Volt erhöht.

Externe Module wie der eingebaute Lautsprecher, das integrierte Mikrofon, die Kombination aus Batterie, Klinkenbuchse und Lademodul sowie die Status-LED werden mit Steckverbindungen an die Platine angeschlossen, um eine einfache Montage zu ermöglichen. Da sich zwischen Gehäusedeckel und Platinenoberseite noch Platz befindet, konnten wir das WM8960 Audio Codec Board auf diese montieren. Die Verbindung zum Raspberry Pi wurde mittels Kabeln direkt zwischen dem 40-Pin-Header und den einzelnen Pins des WM8960 umgesetzt. Die I2S- bzw. I2C-Verbindungen vom Raspberry zum Codec entsprechen denen des WM8960-HATs von Waveshare, wodurch der Treiber von Waveshare ohne weitere Konfiguration nutzbar ist.

Der komplette Schaltplan als Bilddatei ist unter [EyeAIVisionPro/docs/images/EyeAIVision_Pro_HAT_circuit.tif](https://www.waveshare.com/docs/EyeAIVisionPro/docs/images/EyeAIVision_Pro_HAT_circuit.tif) einsehbar.

Gehäusedesign

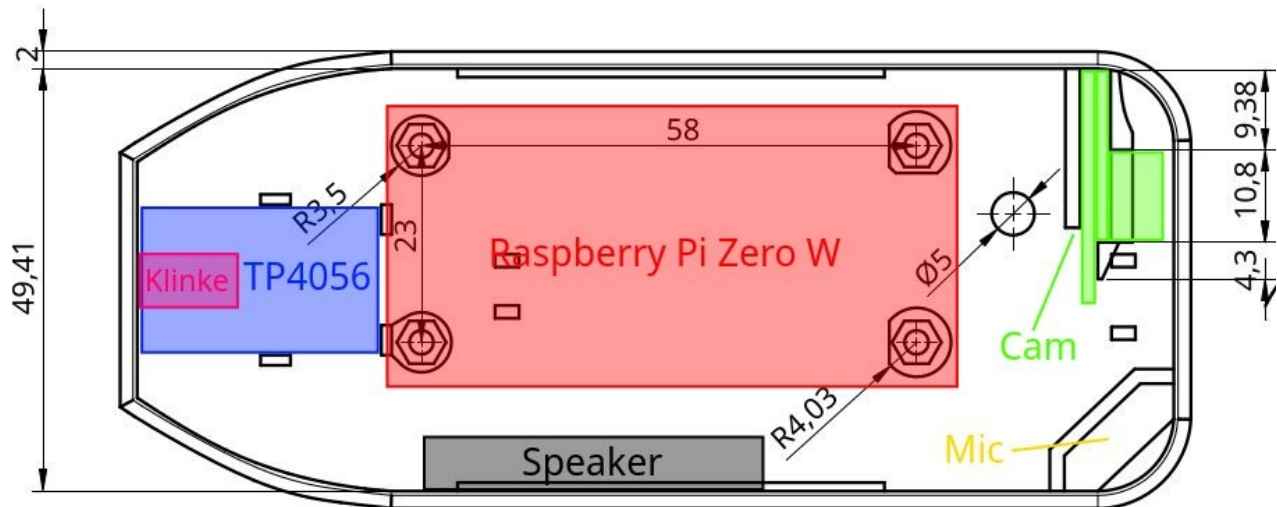


Abbildung 2: Gehäuse mit Platzplan für alle Bauteile

Zur Befestigung des Raspberry Pi und des HATs im Gehäuse befinden sich am Gehäuseboden vier Kunststofffassungen für M2,5-Muttern. Die Muttern können mithilfe von Sekundenkleber in den Fassungen fixiert werden.

In die befestigten Muttern werden anschließend vier M2,5 × 10 mm Sechskant-Abstandsbolzen eingeschraubt, auf denen der Raspberry Pi montiert wird. Das HAT kann anschließend auf die Abstandsbolzen des Raspberry Pi aufgesteckt werden. Da das HAT über vier entsprechende Bohrungen verfügt, werden zur Befestigung weitere M2,5 × 8 mm Abstandsbolzen verwendet.

Im Deckel der EyeAIVisionPro befinden sich vier Senkkopfbohrungen. Durch diese kann der Deckel mithilfe geeigneter Schrauben an den Abstandsbolzen befestigt und das Gehäuse verschlossen werden.

Der Deckel enthält eine größere Bohrung mit zwei Aussparungen, in welcher Platz für die aus Kunststoff bestehende Funktionstaste ist. Diese liegt direkt über dem Taster auf der Platine.

An der Hinterseite des Geräts liegt das TP4056-Lademodul. Dieses wird mit doppelseitigem Klebeband am Gehäuseboden befestigt. Die USB-C-Schnittstelle von diesem zeigt direkt aus dem Gehäuse, um das Einstecken eines Ladekabels zu ermöglichen.

Da die Batterie außerhalb des Gehäuses (z. B. an der anderen Seite der Brille) liegt, ist im vorderen Bereich eine Bohrung mit 5 mm Durchmesser vorhanden. Durch diese kann das Batteriekabel nach draußen verlegt werden.

Über dem TP4056 wird in die Bohrung im Gehäuse eine 3,5-mm-Klinkenbuchse für Kopfhörer eingeschraubt. Das eingebaute Mikrofon liegt an der unteren Kante des Gehäuses und wird in die dafür vorgesehene Tasche eingeschoben. An der Stelle des Sensors befinden sich konzentrisch angeordnete Bohrungen, um die Schallwellen weiterleiten zu können.

Für das Kameramodul haben wir eine Schiebelösung ausgewählt. Durch diese ist es möglich, das Modul ohne Kleben oder starke Krafteinwirkung in die Befestigung zu schieben. Somit wird die Linse direkt an das Kameraloch gehalten.

An der Unterseite liegt der interne Lautsprecher. Für diesen befinden sich in der Wand des Gehäuses Schlitzlöcher für die Schallwellen. Die EyeAIVisionPro enthält rechteckige Löcher am Boden des Gehäuses, durch welche schmale Kabelbinder geschoben werden können, um das Gerät am Brillenbügel zu befestigen.

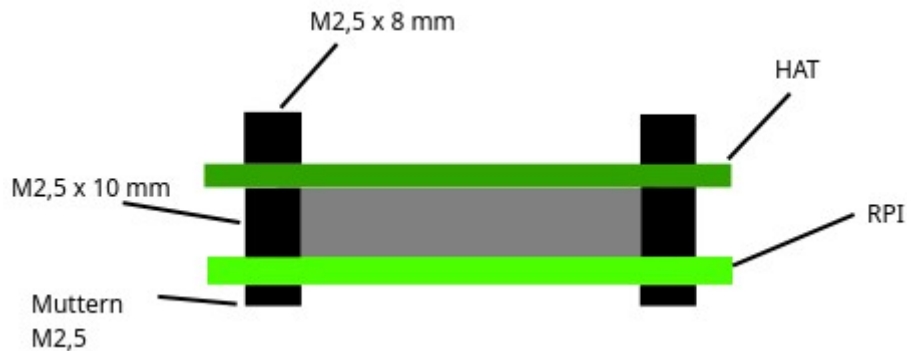


Abbildung 3: Raspberry Pi und HAT im Gehäuse

Software

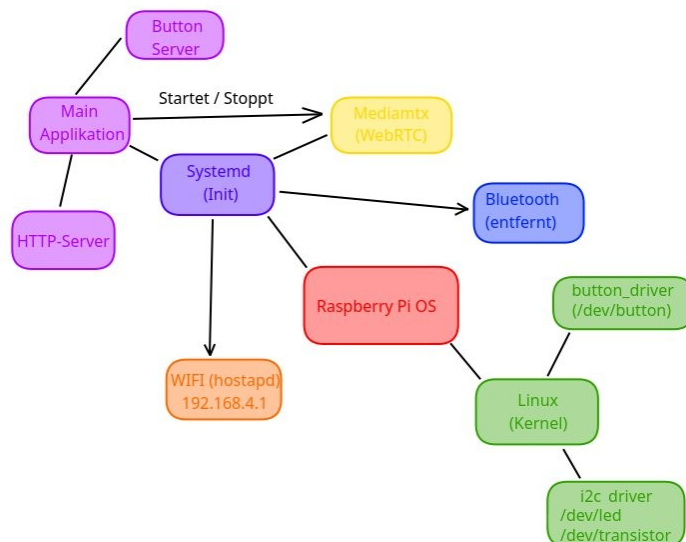


Abbildung 4: Betriebssystem EyeAIVisionPro

Das Betriebssystem der EyeAIVisionPro basiert auf Raspberry Pi OS Lite Trixie, einer Distribution von Debian 13. Damit kann sichergestellt werden, dass alle Gerätetreiber ordnungsgemäß mit dem Raspberry Pi arbeiten. Als Init-System wird das vorinstallierte systemd verwendet. Der Linux-Kernel des Systems ist ebenfalls der von Raspberry Pi OS. Allerdings werden zusätzlich zwei eigens entwickelte Kernel-Module geladen, welche „button_driver“ und „i2c_driver“ heißen. Diese Module sorgen dafür, dass die Funktionalität des HATs per Character Device aus dem Userspace gesteuert und ausgelesen werden kann.

So erstellt `button_driver` unter `/dev/button` ein Character Device, in welchem der Status der Funktionstaste ausgelesen werden kann. Bei gedrücktem Taster wird eine „1“ gelesen, ansonsten eine „0“. Das Modul `i2c_driver` ist für alle Funktionen zuständig, die mit dem ATtiny85-Mikrocontroller gesteuert werden. Dazu zählen derzeit das Kappen der Stromversorgung sowie die Steuerung der Status-LED. Für Ersteres muss in die Datei `/dev/transistor` eine „0“ geschrieben werden. Dies muss aus Sicherheitsgründen jedoch mit Administratorzugriff geschehen. Im Anschluss wird die Stromversorgung unterbrochen, wodurch das System ohne Herunterfahren abgeschaltet wird. Um dies zu vermeiden, erfolgt dies mithilfe des `systemd`-Services `transistor-off.service` genau dann, wenn das gesamte Betriebssystem heruntergefahren wurde.

Um die Status-LED umschalten zu können, muss in `/dev/led` ein „G“ (für Grün) oder ein „R“ (für Rot) geschrieben werden. Für die Kommunikation mit einem Mobiltelefon startet das Betriebssystem einen WLAN Access Point mit der SSID „EyeAIVisionPro“. Der Einfachheit halber während der Entwicklung ist das Passwort fest und lautet „12345678“.

Um die Einsatzbereitschaft des Geräts zu signalisieren, ertönt nach dem Start von diesem ein Ton auf dem internen Lautsprecher, welcher durch den Service `hotspot-sound.service` gestartet wird. Um die EyeAIVisionPro mittels Funktionstaste herunterfahren zu können, startet `systemd` den Service `button-shutdown.service`, welcher ein Skript startet, das nach einem 3-sekündigen Druck Linux herunterfährt und einen Shutdown-Sound abspielt.

Damit das Gerät als Bildquelle für die EyeAIApp genutzt werden kann, haben wir eine Hauptapplikation in QtCore entwickelt. Ihre Aufgabe ist es, alle wichtigen Funktionen zu instrumentalisieren. Dazu gehört in erster Linie, den Media Router MediaMTX zu starten. Dieser unterstützt das WebRTC-Protokoll, welches eine Bildübertragung über das Netzwerk bei geringer Latenz (in unserem Fall ca. 100–200 ms) ermöglicht. Außerdem arbeitet MediaMTX ohne weitere Konfiguration mit dem Raspberry Pi Camera Module 3 und der zugehörigen H.264-Hardwarekodierung. MediaMTX ist als nicht automatisch startender und benutzerspezifischer `systemd`-Service (`mediamtx.service`) des Benutzers „eyeai“ eingetragen. Die Hauptapplikation startet diesen, sobald sie gestartet wird. Beim Stoppen wird MediaMTX ebenfalls per `systemd` beendet. Diese Architektur ermöglicht uns eine Steuerung externer Programme, ohne dass diese direkt von unserer Hauptapplikation aufgerufen werden müssen, was die Robustheit maximiert.

Damit Nutzer die Sprachsteuerung verwenden können, ohne diese in der EyeAIApp zu starten, besitzt die Hauptapplikation einen einfachen TCP-Server, um im Falle eines Tastendruckereignisses dies der App mitteilen zu können. Dazu verbindet diese sich als TCP-Client mit dem Server auf dem Port 3333. Wird die Taste ein- oder mehrfach gedrückt, so erhält der Client die Anzahl an Klicks (zwischen 1 und 3) als Character und kann danach die Sprachsteuerung beispielsweise starten bzw. stoppen. Dieser Socket dient außerdem als Verbindungsindikator mit der EyeAIApp. Ist diese verbunden, so schaltet die Status-LED auf Grün.

Zusätzlich zum TCP-Server besitzt die Hauptapplikation einen HTTP-Server, über welchen sich alle Logs von MediaMTX und der Applikation selbst sowie die Bluetooth-MAC-Adresse der EyeAIVisionPro auslesen lassen.

Eine weitere Verbesserung zum Vorgänger sollte die Möglichkeit sein, das Spatial-Audio der EyeAIApp über den Kopfhörerausgang der EyeAIVisionPro ausgeben zu können, damit separate (Bluetooth-)Kopfhörer, die mit dem Mobiltelefon verbunden sind, nicht mehr notwendig sind. Dazu haben wir vorgehabt, ein Bluetooth-A2DP-Audio-Sink zu starten, über welches die EyeAIApp Tondaten ausgeben kann. Allerdings hat sich in praktischen Tests herausgestellt, dass der BCM2835 SoC des Raspberrys Probleme bei der gleichzeitigen Nutzung von WLAN und Bluetooth hat. Deshalb gab es häufige Verbindungsaussetzer beim Versuch, Video über WLAN und Ton über Bluetooth zu streamen. Aus diesem Grund haben wir diese Funktion entfernt. Eine mögliche Lösung dafür könnte das Audiostreaming über WLAN sein, welches wir aufgrund von Zeitproblemen nicht mehr umsetzen konnten.

Praktische Verwendung der EyeAIVisionPro

1. Durch einen kurzen Knopfdruck auf die Einschalttaste startet die EyeAIVisionPro und die Status-LED leuchtet Rot. Der Gerätestart dauert normalerweise zwischen 1 und 2 Minuten. Sobald der Startton ertönt, ist das Gerät einsatzbereit.
2. Nach dem Start von EyeAIApp sollte dann unter „EyeAI-Vision auswählen“ der Eintrag „EyeAIVisionPro“ ausgewählt werden
3. Anschließend muss „Systemaudiogerät verwenden“ gewählt werden.
4. Bei erfolgreicher Verbindung sollte die Status-LED Grün aufleuchten. Nun sollte das Kamerabild der EyeAIVisionPro zur KI-Datenanalyse verwendet werden.
5. Bei einmaligem Tastendruck auf der EyeAIVisionPro wird die Sprachsteuerung gestartet. Um diese zu Stoppen, muss ein Doppelclick erfolgen.
6. Um das Gerät nach der Verwendung herunterzufahren, muss die Einschalttaste für ca. 3 Sekunden gedrückt werden. Das Herunterfahren wird mit einem Signalton signalisiert.

Bekannte Probleme

- Trotz hardwaretechnischer Möglichkeit ist es noch nicht möglich, das Spatial-Audio über die EyeAIVisionPro abzuspielen.
- Aufgrund von Zeitproblemen konnten wir noch keine Batteriestandsmessung integrieren, obwohl sie hardwareseitig bereits Möglich wäre.

Technische Daten

Name	EyeAIVisionPro
Abmessungen (B*H*T)	120mm * 53 mm * 29 mm
Gehäusematerial	PLA
Akkukapazität	1500 mAh
Schnittstellen	1x USB-C, 1x 3,5 mm Klinke

SoC	BCM 2835 SOC @ 1GHz, 512 MB RAM, Raspberry Pi Zero W
Kamera	Raspberry Pi Camera Module 3 Wide
Kamerachip, Max. Auflösung	Sony IMX708, 11,9 MP
WebRTC Videoauflösung	640x480 (VGA)
Kamera FOV (Max.)	120°

Fazit

Mit der EyeAIVisionPro ist es uns gelungen, ein Gerät zu entwickeln, dessen Hauptzweck darin besteht, einen stabilen Videostream aus der Perspektive des Geräts auf das Mobiltelefon des Nutzers zu übertragen. Allerdings konnten wir nicht alle Features (wie z.B. die Tonübertragung) aufgrund von Zeitmangel rechtzeitig integrieren.